The Measurement Book

Laws of the SIGIL Graph: the measured record

A companion to *SIGIL — The Variational Chain*, whitepaper v1.3

v0 · 2026-07-24

`bitknight.dipper688@passmail.net`
SIGIL / Flux Foundation · `sigilgraph.quillon.xyz`

"When you can measure what you are speaking about, and express it in numbers, you know something about it; when you cannot measure it . . . your knowledge is of a meagre and unsatisfactory kind."

— William Thomson, Lord Kelvin, 1883

Scope & method. This book compiles every measurement arc in the SIGIL record into one document: what was measured, on what instrument, with what result, and — just as load-bearing — what each measurement *ruled out*. Nothing here is new work; everything is sourced from benchmark logs, journal excerpts, commit messages, and the project's measurement notes, with exact numbers kept verbatim. Where the whitepaper's §3 (*Measured laws*) gives the summary, this book gives the method, the residual tables, and the failures of the instruments themselves. Two standing rules bind every page: **benchmark numbers keep their host** (a 48-core figure is never transplanted onto a phone), and a claim's **evidence grade travels with it** (see §1). All measurements were taken on one operator-funded fleet — a household-scale lab, not a public network; legibility, not scale, is what this record buys.

15
MEASUREMENT ARCS

2
LAWS WITH RESIDUALS
(0.4 pp sim · 1.50 pp live)

5
PARTS: METHOD · LADDER · LAWS · FORENSICS · NEGATIVES

● MEASURED ⊕ DERIVED+SIM ⊙ SIMULATION ◇ STAGED ○ OPEN △ *conjectural*

Markers are read as two axes — *evidence grade* (derived / simulated / benchmarked / live-measured) and *deployment grade* (live / staged / dormant / absent). DERIVED+SIM names the coordinate a claim holds when it is mathematically derived and validated in seeded simulation but not yet measured on a real network. Equations are set in black; color is reserved for results and status.

1 Method: how this record grades itself

The SIGIL project's development methodology (Variational Flow) requires every claim to pass a falsifiable measurement gate before it is called done, and every measured number to carry two independent coordinates:

- **Evidence grade** answers *"how do you know?"* — one of DERIVED, SIMULATED, BENCHMARKED, LIVE-MEASURED.
- **Deployment grade** answers *"is it running?"* — one of LIVE, STAGED, DORMANT, ABSENT.

A single ladder conflates the two; the cardinal sin is the category error — "passed tests" and "mathematically safe" are different sentences. The canonical worked example is the delivery law itself (§3.1): first graded MEASURED on simulation evidence, then *regraded* to DERIVED+SIM when review caught that the validating simulator shared the law's own independence assumption, and only promoted back to MEASURED after a real-kernel network experiment. Regrades are logged, not silently absorbed.

1.1 Rule 0: measure the live path first

The single most expensive class of error in this record is the *microbench beside a slow live number*. The sync ladder (§2.1) contains the canonical case: a flat store measured at ∼10,000,000 records/s durable while the live node synced at ∼800 blocks/s — four orders of magnitude apart, and the gap was *not* in any component that had been benchmarked. A fast component beside a slow system is a bug report, not a victory. Every arc in this book therefore states which rung of the microbench → harness → live ladder its numbers live on.

1.2 Instrument calibration: when the measuring tools lied

A measurement record is only as good as its instruments, and the instruments failed repeatedly. These are recorded as calibration hazards, since several "results" in earlier working notes were artifacts of them:

- **The 0/0 test trap.** The build orchestrator's combined test tool reported *0 passed, 0 failed* when the test binary failed to compile — readable as "no failures." A stale server process holding a deleted executable produced the same signature. Rule: gate on an explicit verdict, never on `failed==0`.
- **Stale binaries.** `ls -t` on a shared `deps/` directory selected a three-week-old test binary; the "result" measured old code. Fixed by a tool that prints the exact fresh binary per target.
- **Swallowed filters.** The test runner silently dropped `--ignored <filter>` arguments, so an expensive benchmark that appeared to run in 0 s had in fact not run at all (fixed in commit `792272ec`).
- **Self-reporting caches.** A storage cache reported its own hit-rate as victory while an independent probe measured 0.4% of reads returning data. Latency probes that time `get()` without asserting the value came back measure nothing. Rule: any "it's fast now" claim must be paired with an independent correctness check on the same artifact.

> IN PLAIN WORDS — **Why a book of numbers needs a chapter about broken rulers**
>
> Before trusting any measurement in this book, the project had to learn — the hard way — that its own stopwatches and test tools sometimes reported success without doing the work. Each of those failures is listed here so a reader can check that the numbers that follow were taken after the instruments were fixed, not before.

2 Part II — The performance ladder

The ladder runs microbench → in-process harness → live node. Each rung below carries its negative results, because the negative results are what directed the work: five of the six "obvious" optimizations measured as no-ops.

2.1 The sync ladder and the honest bottleneck

ARC-5 · Block-sync throughput, wall by wall 2026-06-19 → 07, epsilon 48-core ● MEASURED

MEASURED Durable commit throughput of the LSM key-value store: **~3,863 blk/s** ceiling (fsync ON 3,268 / OFF 3,548; batch 4,096 vs 16,384 identical). Root cost ~130 μs per KV entry inside `batch_put` (WAL + memtable insert + SST flush). Bench: `SIGIL_COMMIT_BENCH=2000000 ... commit_sink_throughput`.

FIX 1 Flat append-only `SkeletonStore` (72-B records: height‖hash‖parent), offset = $(h - \text{base}) \cdot 72$, two-phase durable write, torn-tail truncate on open: **~10,000,000 rec/s durable** — 2,600× the KV path (commit `615351c`, later consolidated as `flux_db::skeleton`).

FIX 2 SST bulk-ingest engine (build sorted SST bytes, atomic tmp→fsync→rename install, skipping WAL/memtable/compaction): **238,762 blk/s** release (debug 184,825) vs batched-KV 110k on the same host — dark behind `SIGIL_DB_SST_INGEST`.

HARNESS Permanent in-process end-to-end harness (`SIGIL_SNAP_BENCH=N`; snapshot pull + stream verifier + commit hook vs synthetic codec=2 server, 2M records, roots match): **39,112 blk/s** (commit `f52373d`). Next wall identified: per-record bincode round-trips, ~25 μs/record.

LIVE The live node sat at ~800 blk/s for five days *despite all of the above*. The actual caps were producer-side: a full-block serve throttle (≤1 per 120 ms) and a serial request→await→apply loop. Removing both (commit `91e4598`: memcpy cache-serve of finalized ranges + request-ahead window ~16 ranges): **2,322 → 10,853 blk/s** on the running node. Pre-sprint live WAN baseline: 96–144 blk/s. Idealized transport ceiling from the deterministic simulator: 92,600 blk/s.

RULED OUT Per-block GETs as the wall (trusted bulk = +15% only); the read-dir cache on a fresh DB (0 effect); fsync on/off (±8%); batch size (0); parallel hash/serialize (already saturated); the wire and the fold fast-path (never the wall); per-write fsync as the compaction cost (WAL has none — the wall is synchronous leveled compaction, ~10–20× write amplification).

OPEN Snapshot-pull (the 100× path) is coded and byte-symmetric but gated OFF pending a published signed anchor. Sustained ≥92.6k live has never been demonstrated. ◇ STAGED

2.2 Wire compression

ARC-6 · zstd on the header wire 2026-06-10, epsilon, live ● LIVE

MEASURED Candidates timed on a *real* 4,096-header chunk dumped from the monitor DB (raw 1,019 B/header): lz4-1 = 10.95× at 20 ms; **zstd-1** = 14.03× at 20 ms compress / 12 ms decompress. Gossip lane, a realistic hex-proof header: **26.3×** (6,973→265 B, 96.2% saved).

LIVE Deployed as backfill codec=1; live journal shows "served 4097 HEADERS … 289771 B, codec zstd" = **70.7** B/header to an off-box client. At that size a 1 Gbit link carries ~1.7M headers/s — the wire stopped being the sync bottleneck. Same-day verify fixes: stored-hash 32-byte compare verify **84k blk/s under loadavg 65** (was 26–52k); parallel verify later ≈160k blk/s.

RULED OUT lz4 beating zstd at these chunk sizes (the operator's hunch, measured wrong); changing the header hash encoding as a monitor-side fix (the canonical-JSON hash *is* block identity — only a height-gated chain upgrade can touch it).

RECORD Decompression bomb cap 64 MB; client side pure-Rust decoder for Windows-clean cross builds; interop gate: C encoder → Rust decoder test.

2.3 Throughput ceilings: what the box can and cannot do

ARC-11 · Verified-TPS ceilings, bench vs reality 2026-05-30 → 07-04, 48-core + 4-node fleet ● MEASURED

LIVE — 4-node verified-transaction TPS, direct transport: **80,102 TPS** aggregate (epsilon 46,824 + delta 19,189 + gamma 5,371 + beta 8,718). Per box: ed25519 verify-once ~800k sig/s; state fold free at 209M commits/s (not the wall).

BENCH VS REAL — Batch authorization standalone: B= 1 → 251k, B= 4096 → **152M TPS/box** (190×, state-fold-bound). The *real* pipeline (apply + commit + 4 roots, 10k authors): B= 1 → 12,352; B= 1024 →**98,470 TPS**, plateau ~98k. The 152M does **not** survive contact with the real pipeline — next walls are per-op apply (~4 μs) and naive O(state) root recompute.

ZK TAX — ed25519 inside a STARK circuit: 363 ms at 2^{14} scale vs 1.25 μs native ≈ **290,000×**. Dilithium5 in-circuit 876,830 constraints (only 3.4× cheaper than ed25519's ~3M). Proof aggregation gives an $O(1)$ verifier — a 64k-tx block via one 50 KB proof in 0.06 ms — which is a light-client lever, **not** a throughput lever: the zk *produce* ceiling measured ~59–65k TPS, linear in trace length.

LADDER — TPS-ladder foundations (2026-07-04): per-tx signed 15,954; batch@64 102,630 (degrades past 512); sequential full apply 108,696; sharded apply N= 16/N= 64 = **1,006,289 / 1,921,922 TPS** on one box (wind-tunnel, not live); replica validators *anti*-scale 84k→2.9k (~ $1/N$).

RULED OUT — zk-STARKs as a throughput mechanism; replica validation as a scaling mechanism; hash speed as a state-root lever (the $O(1)$ accumulator made it moot).

OPEN — A soundness finding rode along: the authorized-batch path had no nonce — replay after producer restart re-executes. A live soundness gap independent of TPS, tracked in the hardening backlog. ○ OPEN

2.4 The producer crawl

ARC-12 · 0.5 blk/s → 1,230 blk/s, live — 2026-06-08, epsilon ● LIVE

MEASURED — Producer decayed to ~0.5 blk/s at ~27k blocks. Two $O(N^2)$ leaks: (1) a full-chain JSON snapshot every 100 blocks — deterministic replay at 25k blocks: old = 250 saves / 389 MB / 47.8 s vs time-gated = 1 save / 3.1 MB / 0.4 s (**126×**, growing with N); (2) a backfill-serve storm (513-block chunk re-broadcasts, 12× per 20 s, ~6k blk/s serialized, 84% CPU). After both fixes: **~1,229 blk/s** live (commit `e34ad15`).

NEGATIVE — A simulator-tuned drain-fast setting (250 ms) lifted *modeled* catch-up 14× — and collapsed *live* production to 0 blk/s at 82% CPU with a single backfiller. Gossip-rebroadcast backfill is architecturally dead: serve load scales as rate×(N+1). The simulator modeled the timing race, not the gossip amplification. Validate tuning live before trusting the model.

FIX — Point-to-point request-response backfill (chunk = 1,024; a 4,096-block ~2 MB response silently exceeded the ~1 MB protocol limit): fresh node closed a 16k+ gap while the producer held ~1,380 blk/s; 4-node mesh, 3 backfillers, 0 divergence at ~900 blk/s.

FOLLOW-UPS — Producer OOM (1,075 crash-loops) → memory-bound append-only `chain.log` with an 8,192-block RAM window: RAM plateau ~35 MB while the log exceeded 257 MB; restart stream-recovered 102,548 blocks. Boot replay of a 52 GB log costs ~35 min — the motivation for snapshot boot. ○ OPEN

IN PLAIN WORDS — **What the ladder teaches**

Making one part of a system a thousand times faster often changes nothing, because the slowness lives somewhere nobody benchmarked. The project measured its storage at ten million records per second while the real node crawled at eight hundred — the fix that finally mattered was in how the *serving* node queued its answers. That is why this book insists on saying, for every number, whether it was taken on a test bench or on the running machine.

3 Part III — Physical laws

3.1 The delivery law: simulation

ARC-1 · $D = 1 - p^r$ under seeded simulation 2026-06-16/17, deterministic simulator ⊙ SIMULATION

CLAIM Unique-delivery fraction under star-flood gossip with redundancy r and i.i.d. drop probability p follows $D = 1 - p^r$; provisioning inverse $r^* = \lceil \ln(1 - \text{SLA}) / \ln p \rceil$.

METHOD 22 seeded simulator runs (1 producer → $N-1$ sinks), $N = 4/8/16$, virtual time. Reference cell: 16 nodes, 200 msgs (3,000 expected deliveries), 50 ms edge latency, seed 42.

RESULT r=1, p=0 → 100.0% (27 ms wall); r=1, p=0.3 → 69.8% (2,094/3,000, byte-identical on re-run); r=3, p=0.3 → 97.4%; r=5, p=0.3 → 99.7%. **RMS residual 0.4 pp** (within 1 binomial σ). Shipped default r=3 holds 99% delivery up to $p \leq 0.215$.

RULED OUT Node-count dependence (N = 4/8/16 identical); nondeterminism (the exact same 2,094 messages delivered twice).

REGRADE Review caught that the simulator *implements* the law's independence assumption — circular validation. The claim was regraded DERIVED+SIM in whitepaper v1.2, which forced the promotion experiment below. The regrade is part of the record, not an embarrassment to be edited out.

3.2 The delivery law: promotion on a real network stack

ARC-2 · netem promotion experiment 2026-07-22, 8 netns peers, kernel netem ● MEASURED

METHOD A dedicated probe crate speaking the real backfill request/response protocol (responses 70,700 B ≈ one 1,000-header chunk at the live 70.7 B/header). 8 peers in Linux network namespaces on a private bridge, kernel netem loss both directions. Delivery = at least one reply within the shipped 10 s timeout. Conditions $\times\{r{=}1,2,3\} \times 600$ trials: baseline 20 ms; +10%, +25%, +40% i.i.d. loss; Gilbert–Elliott burst loss (≈30% avg, mean burst ≈2.9 packets — the pre-registered falsification condition). Totals **9,000 trials / 18,000 attempts**. $\hat{p}$ is measured per run, never assumed; r=1 cells are calibration.

RESULT +10%: $\hat{p} = 0.0000$ over 3,600 attempts — TCP absorbs ≤10% packet loss entirely, so *the law's p lives at the request level, not the packet level.* +25%: $\hat{p} = 0.055$, residuals +0.13 pp (r=2) / +0.02 pp (r=3). +40%: $\hat{p} = 0.873 \to 0.942$, residuals −0.76 / −3.02 pp (transport collapse, sub-independence in the predicted direction; the only cell ≈ 2.2σ). Burst: $\hat{p} = 0.512$–0.62, residuals +1.27 / +1.47 pp — burst loss does *not* break cross-peer composition. **Within-run RMS 1.50 pp** over the 6 informative cells; max |residual| 3.02 pp; binomial 1σ at n=600 is 0.9–2.0 pp. Promotion to MEASURED recorded in commit `73bac94`; whitepaper → v1.3.

RULED OUT Packet-level p as the law's variable; burst loss breaking composition; r=3 as a safety mechanism at 40% both-direction loss (nothing at the gossip layer is); per-peer product refinement materially beating the pooled law (peer $\hat{p}$ spread 0.52–0.72 is second-order).

OPEN WAN replication attempted 2026-07-23 and **blocked** — two fleet boxes down, one decommissioned. The portable probe kit is staged. Namespaces are not a WAN; the promotion holds for a real kernel stack on one host. ○ OPEN

3.3 The fold: constant-size whole-chain verification

ARC-4 · Fold proof size and verify cost 2026-07-18 re-measured, 48-core (loaded) ● MEASURED

RESULT Proof **2,568 B constant** for any chain length; full verify **280–342 ms** at 100k blocks; incremental tip check vs an already-verified fold state **3 µs**; live verifier downloads 0 blocks. In-sim variant: 392 B constant for 20 vs 2,000 blocks of history; late-join run skipped 165 historical blocks, applied 20 live, divergence = 0.

CAVEAT Verification is $O(M)$ — it reads all M per-block commitments (∼51 MB at 100k blocks). A constant-size *proof* is not constant-cost *verification*, and 342 ms belongs to a loaded 48-core host — it is not a phone number. Folding the commitment vector is the open v0.3 lane.

RULED OUT That the 342 ms figure was documentation vaporware — the hardening workstream re-ran the bench and it held (the whitepaper had *understated* maturity).

OPEN Full verify of the live ∼29M-block fold never separately benchmarked; the lattice-reduction security argument has no published concrete-parameter proof; in-sim snapshot adoption is modeled (producer clone), not peer-fetched. ○ OPEN

3.4 Mining physics: units, levers, and an anti-parallel proof

ARC-14 · Φ, Ω, and the crypto benches 2026-05-31 → 06-11, 48-core (loaded) ● MEASURED

HASH Parallel-sound hash lane: **155 MH/s** on 48 cores (5 MH/s on one, ∼31× scaling); invertible turbo variant (ceiling only) **12.9 GH/s**. Units locked: 1 Φ ≡ 1 EH/s, 1 nΦ = 1 GH/s. The faster hash is an **83×** lever for *mining* and ∼0× for *state roots* — the $O(1)$ accumulator made root hashing moot.

VDF 2048-bit Wesolowski VDF: single-core **116.4 mΩ** (116,374 turns/s); 48 parallel chains still ∼29.5 mΩ *each* — more cores buy **zero** speedup for one VDF. That anti-parallel behavior is the measured result; the absolute mΩ figure is pure-Rust and host-bound. $T = 10^6$ gives an 8.6 s delay.

PQ ML-KEM-1024 hybrid channel (encapsulation over noise, HKDF + per-direction AEAD), browser↔bridge headless end-to-end: **115 ms**, shared key verified on both ends. Negative finding riding along: the wallet's pre-existing "PQ connection encrypter" was **pretend** — it never encrypted and never compared transcript hashes.

TIP Browser-wasm tip check of the *live* chain tip ($h = 742{,}989$): **0.10 ms**, ok:true; tampered height or root byte flip → rejected. Honest gap: the fingerprint flavor is a corruption check, not forging-adversary-resistant — the PQ-signature variant is blocked from wasm by a dependency. ○ OPEN

CALIBRATION A signature-size overclaim was caught by measurement: the PQ signature crate claimed Level-5 output while its keygen defaulted to Level 1 (148 B); the fix produces the true 292 B Level-5 signatures.

3.5 Braid convergence: from simulation gate to live byte-identity

ARC-7 · DagKnight braid, three phases 2026-07-02 → 07-24, sim + 2-node live ● MEASURED

SIM GATE Six adversarial scenarios, all green: 10k blocks / 3 producers, divergence = 0; 30% drop/reorder/equivocation, divergence = 0; 16/16 orderings agree, incremental ≡ batch; withheld-block attacker's finalized prefix byte-identical over 1,378 blocks; 5/5 tampers rejected; 100k blocks, max reorg window 258 ≤ 16,384 bound. Graded SIMULATION in the whitepaper — correctly, as the next row shows.

SETTLE Settlement throughput bench: 1M transfers through the state-commit path = **67,248 tx/s** in a DEBUG build (release estimated 300k–1M, hash-bound). Ruled out: settlement as the wall — the live chain is block-production-starved, not settlement-bound.

LIVE First live 2-node run: the weave was real (1,856/4,642 blocks carried merge parents) but **money diverged** — supply 2,319,501,400,000 vs 2,320,001,400,000 units across nodes; one node read the other producer's balance as 0. Root cause: settlement drained the *local* tip, not the braid spine. Fix (commit `5d67d25`, three coupled bugs): both nodes byte-identical on supply and balances, staying identical across samples as supply grew 3.142T→3.146T. Critical constant: finality depth 64 (∼10 s at 150 ms blocks) ≫ gossip delay; depth 4 wedged permanently. Coinbase gate: 4-node deterministic replay of 200 coinbase blocks → identical wallet root + supply, and supply = ∑rewards exactly; solo braid, 60 blocks: producer balance 295.0 = 59 × 5.0 (emission lags one block).

OPEN No safety/liveness proof — "deterministic braid linearization v1," not the published DAGKnight algorithm. Known P0: a reorg below the finalized line permanently wedges settlement (append-only tip, no rollback). Transactions-per-block and full-block propagation under the braid are unmeasured — the 100+ blk/s figures were *empty* blocks. ○ OPEN

4 Part IV — Diagnosis stories: measurement as forensics

4.1 The propagation chain

ARC-13 · Loopback bisection → WG proof → 10M-node flood 2026-05-31 → 06-01 ● MEASURED

BISECT 1 producer + 4 followers on loopback ($0 experiment): only 1/4 followers received blocks; the producer showed 11 connect / 11 disconnect churn. Root cause: all followers derived *one deterministic peer identity* from the same node-id string — the transport deduplicated N processes into one logical peer. Fix (pid-suffixed ids): 4/4 followers, blocks_applied = 60 each, divergence = 0. Each alternative was ruled out explicitly: idle-timeout (already 60 s), NAT (reproduced on loopback), protocol mismatch (Identify matched), explicit disconnects (none in code).

2ND CAUSE Gossip has no history: a late joiner received 114 blocks, applied 0, rejected 114 — chained to a height it could never reach. A design gap, not a bug; it became the request-response backfill (§2) and the fold late-join.

WG First cross-server transport proof over WireGuard: blocks_applied = 50, divergence = 0, rejected = 0; ping 0.585 ms RTT; 1.24 GiB transferred. The real bugs found were tooling (a stale journal peer-id; follower must graft first) plus one transport bug: the Tor stream's write path never flushed.

FLOOD Deterministic flood simulator, uncapped: 100K nodes → 100.00% delivery, 124 ms, 58 MB; 1M nodes → 100.00% (999,999/999,999), 1.44 s, 520 MB; 10M nodes → **100.00%, 23.9 s, 5.2 GB**, all deterministic. Ruled out: the "10M ceiling" (an artifact of a 150 s timeout, not the engine).
⊙ SIMULATION

4.2 The GPU hashrate collapse

ARC-8 · 3 GH/s → 300 MH/s, predicted and matched 2026-07-24, live rig + code ● MEASURED

MEASURED A GPU rig's rate collapsed from ∼3 GH/s to 150–300 MH/s. Reading the miner's thermal guard gave a quantitative prediction: throttle at ≥78 °C adds 50 ms sleep per dispatch; default batch 16.7M nonces at ∼5.6 ms kernel time ⇒ 16.7M/55.6 ms ≈ 300 MH/s — the live rig reported **297 MH/s**. Diagnosis by arithmetic, confirmed by telemetry.

SECONDARY An env var was read twice with different defaults (dispatch sleep 0, thermal sleep 50); the "CPU flicker" was a measurement artifact (16-bit share target → millisecond timed slices, no smoothing); and the 64-shares-per-height wallet cap against a GPU producing ∼45k share-grade nonces/s cut the effective GPU duty cycle to ∼2–4% (785 wallet-cap rejects live).

FIX Commit `f34d06c`: thermal defaults raised to 85/90/80 °C with the env split; hashrate windowed ≥2 s; *vardiff* (target ∼0.5 shares/s, credit weight $2^{\text{share\_ease}-\text{ease}_w}$). Deployed to the live pool: wallet-cap rejects **933 → 0**; the GPU rig now mines 23-bit shares at weight 128.

RULED OUT A GPU/OpenCL bug; the pool; the share-accounting path.

OPEN The miner-side fix reaches rigs only with the next signed release. ◇ STAGED

4.3 Pool shares: two reject storms

ARC-9/10 · 12k rejects → 15; 99.7% stale-height → 0 2026-07-21 → 23, dev + live pool ● LIVE

E2E Pool end-to-end on a dev node (eased difficulty, two CPU wallets, 64 s): 11 blocks, 497/466 shares banked, ∼2% rejects; the miner that won *zero* blocks still earned 91% of the block-winner's balance — proportional pay measured, not asserted. Conservation (including integer remainder) and solo-degeneration unit-tested (52/52).

LESSON 1 A fixed 150 ms post-share sleep collapsed hashrate → blocks stalled → cap hit → **12k-reject storm**; removing the sleep and switching cap→block-hunt cut it to **15 rejects** in the same test. A politeness delay became a denial-of-service against the miner itself.

LESSON 2 The winner's block solve must credit 1 ≪ ease shares (true work weight); with the naive +1 a share-less pre-7.1 solo miner would earn ∼1/257 of its own blocks.

LIVE 1 Reject-reason counters built and hot-swapped into the live pool: all 3 rigs reported an old client version, 0 shares banked, **100% of 386 rejects = stale height** — the rigs were solo-racing the pool. Control: a current client against the *same* node banked 447 shares at ∼1% rejects. Ruled out: pool bug, GPU share path.

LIVE 2 Deeper cut: stale-height was **99.7% of *all* submit rejects (3,093/3,102)** — at ∼2 s blocks any boundary-straddling share dies. Fix: verify height−1 submits against the snapshotted previous challenge and credit them into the current window (replay-blocked). Validated live within 3 minutes: **229 shares credited, 0 stale-height, 1 benign duplicate**. Server-side only; no consensus change.

RECIPE The 30-second diagnostic that found it: run a known-good CPU miner against the live pool under a pty, then read per-wallet reject counters — verify-mismatch present = real bug; stale-only = client version. ● MEASURED

4.4 The five-minute no-op build

ARC-15 · Depinfo poison: 215.9 s → 0.56 s 2026-07-21/22, build system ● MEASURED

MEASURED "No-op" builds of the chain workspace never no-oped: 215.9 s for an unchanged crate; 166 dirty units / 359 s for the fattest binary. Root cause: cache-restored units skipped the compiler, so a fingerprint dep-info file was never written; the build tool marked those units permanently stale — **19 poisoned roots cascading into 147 downstream units**.

FIX Capture, require, and materialize the dep-info blob on cache restore (self-healing: legacy entries miss once, then re-store complete). Measured heal: 14.8 s→0.56 s; 40.7→2.50; 39.2→6.58 (0 dirty); re-verified next day at **1.0 s** no-op.

RULED OUT Two long-held beliefs died under measurement: the compiler wrapper being hashed into unit identity (the "two fingerprint universes" story — wrong), and the shared cache being empty (a symlink that a disk-usage probe reads as 0 — 17 GB behind it).

RESIDUAL Known secondary taxes, measured and accepted: a release version bump mints new unit metadata (first 1–2 runs legitimately slow); check-family vs test-family profile hashes are distinct universes.

5 Part V — Negative results and open instruments

5.1 An unresolved live datum

ARC-3 · Live-producer serve flakiness 2026-07-22/23, loopback vantage ○ OPEN

MEASURED	The live producer (chain ≈31.6M) served established clients continuously (≈2.1 MB/s, ≈95% serve-cache hit) and bootstrapped one external fresh client from genesis — yet probe clients were served on only ≈2 of ≈35 attempts ($\hat{p} \approx 0.94$ failure), and a fresh local client with a valid handshake got 0 serves and 0 peer-height gossip in 60 s.
RULED OUT	(journal-verified) Request shape; range keys (exact cache-key requests went unanswered); handshake enforcement (log-only); a stale binary (the running executable matched disk); gossip-firehose interference.
NOT RULED OUT	Loopback-vs-public-interface vantage; event-queue contention on a busy producer; response-path loss to short-lived clients. **Not root-caused.** This entry exists because the record keeps its anomalies; the book would be lying by omission without it.

5.2 The ruled-out compendium

Collected from the arcs above — the claims this record has *killed*, kept in one place so they stay dead:

- **zk-STARKs as a throughput lever** — 290,000× in-circuit tax; produce ceiling ∼59–65k TPS. They buy light clients, not TPS.
- **Replica validation as scaling** — measured *anti*-scaling, $\sim 1/N$.
- **Gossip-rebroadcast backfill** — collapsed live production to 0 blk/s with one backfiller; architecturally dead.
- **The 152M TPS bench** — 98k in the real pipeline. Standalone benches do not survive contact with apply loops.
- **lz4 over zstd** for header chunks; **fsync/batch-size/per-block-GET tuning** as sync levers; **node-count dependence** of the delivery law; **packet-level** p as the law's variable; **the 10M-node sim ceiling** (a timeout artifact); **hash speed as a state-root lever**.

5.3 The pretend inventory

Standing items that are built or claimed but *not* measured or not live, graded honestly (the Adversary's Companion reads this same list from the attacker's chair):

- Snapshot-pull fast sync: coded, byte-symmetric, gated OFF pending a signed anchor. ◇ STAGED
- SST bulk-ingest: measured 238k blk/s, dark behind its flag until divergence = 0 on the integrated tree. ◇ STAGED
- Producer-signature enforcement: verifier written and unit-tested, dormant at an unreachable activation height. ◇ STAGED
- Full fold verify of the live ∼29M chain: never separately benchmarked. ○ OPEN
- Braid transactions-per-block and full-block propagation: unmeasured (live figures were empty blocks). ○ OPEN
- WAN delivery-law replication: kit staged, fleet unavailable. ○ OPEN
- The wallet's legacy "PQ connection encrypter": pretend — documented as such until replaced. ○ OPEN

IN PLAIN WORDS — **Why publish the failures and the not-yets**

A benchmark that only reports victories is advertising. The project's rule is that a number is only trustworthy if you can also see the numbers that came out wrong, the optimizations that did nothing, and the list of things still unmeasured. This section is that list, kept current on purpose.

5.4 Limitations of this record

Three limits bind everything above. **One lab:** every number was taken on one operator-funded fleet — mostly one 48-core host — by the people who built the system; there is no external replication and no independent auditor of the grading. **Hosts travel with numbers:** figures like 342 ms, 155 MH/s, and 10,853 blk/s are properties of specific loaded machines, not of the algorithms in the abstract. **Instruments were sometimes wrong:** §1.2 lists the cases we caught; the honest inference is that some were not caught. The mitigation is the method itself — seeds, raw logs, and per-arc provenance sufficient for any reader to re-run what this book claims.

v0 — 2026-07-24. A number without its method is a rumor; a method without its residuals is a brochure. This book is the project's answer to both.

Sources

- *SIGIL — The Variational Chain*, whitepaper v1.3 (2026-07-22) — the canonical design; its §3 *Measured laws* is the summary this book expands. https://quillon.xyz/downloads/SIGIL_WHITEPAPER_v1.pdf.
- *What the Chain Knows — The Philosophy of the SIGIL Graph*, v0.3 (2026-07-22) — the epistemology of honesty; defines the two-axis evidence grading this book applies. https://quillon.xyz/downloads/SIGIL_PHILOSOPHY_v0.pdf.
- *The SIGIL Failure Atlas*, v0 (2026-07-23) — 71 failures in ten classes; its measurement-failure class is §1.2's source material. https://quillon.xyz/downloads/SIGIL_FAILURE_ATLAS_v0.pdf.
- *The Builder's Chronicle*, v0 (2026-07-23) — development settled on-chain as a prototype labor institution. https://quillon.xyz/downloads/SIGIL_BUILDERS_CHRONICLE_v0.pdf.
- *The Adversary's Companion*, v0 (2026-07-24) — the threat model; reads §5.3 from the attacker's chair. https://quillon.xyz/downloads/SIGIL_ADVERSARYS_COMPANION_v0.pdf.
- *The Gossip Delivery Law* (2026-06-17) — the original simulation-campaign note. https://quillon.xyz/downloads/sigil-top-delivery-law.pdf.
- In-tree measurement records (no URL): the delivery-law live kit (`docs/research/delivery-law-live/`: `MEASUREMENT.md`, `summary.txt`, `WAN-STATUS.md`, raw per-attempt `.jsonl.gz`); `docs/THROUGHPUT_MASTER.md`; `docs/SIGIL-TPS-LADDER.md`; `docs/CHRONOS_BENCHMARK.md`; `docs/SIGIL_HARDENING_BACKLOG.md`; and the project's dated working notes, from which all numbers above are quoted verbatim.
- *The Legibility Dividend: An Executive Model of Verifiable Software Production*, v0 (2026-07-26) — the seventh companion: prices this corpus's measurements as a business case (build-latency dividend, Verification Information Yield, the failure taxonomy as a budget allocator, provenance as an audit asset). https://quillon.xyz/downloads/SIGIL_LEGIBILITY_DIVIDEND_v0.pdf.